Software Requirements Specification

IRO | Issues Reporting Outlet

Mini Web-Based Issue Reporting System for AAUA Students and Host Communities

Version 1.0 | 19 October 2026 | Academic Prototype

Prepared for: Software Engineering Project Submission

Document control

Item	Details
Project	Issues Reporting Outlet (IRO)
Platform	Responsive PHP web application
Implementation baseline	IRO-mini.zip, PHP + SQLite
Status	Prototype; not an official or operational complaints service
Primary audience	AAUA students, Akungba, Iwaro, Etioro, Ayegunle and nearby residents

1. Introduction

1.1 Purpose

This SRS describes the actual features, component interactions, constraints and quality requirements of the IRO mini application. It distinguishes implemented functions from proposed future enhancements.

1.2 Problem statement

Students and host-community residents may face fragmented channels for reporting security, accommodation, sanitation, harassment and infrastructure issues. IRO demonstrates a single entry point for recording an issue and checking its recorded status by reference.

1.3 Scope

Included: public home/about sections, issue submission, random tracking reference generation, persistent report storage and reference-based status lookup. Excluded: user registration, administrator login/dashboard, notifications, escalation, actual investigation and resolution workflows. All newly created reports remain Pending in the current application.

1.4 Definitions

Term	Meaning
AAUA	Adekunle Ajasin University, Akungba-Akoko

IRO	Issues Reporting Outlet
SRS	Software Requirements Specification
CSRF	Cross-Site Request Forgery
Tracking reference	Random IRO-prefixed code used to retrieve limited report metadata
SQLite	Embedded relational database used by the prototype

2. Overall description

2.1 Users and operating environment

Public users are AAUA students, indigenes and residents. No login is required. The prototype runs on PHP 8.0+ with the PDO SQLite extension and is viewed in a modern desktop or mobile browser. The PHP process requires permission to create/write iro.sqlite.

2.2 Assumptions and dependencies

- Users have internet or local-network access to the hosted PHP application.
- The server has PHP, PDO SQLite and a writable project directory.
- The system is an academic demonstration; it does not promise action by authorities.
- The tracking reference must be retained by the reporter; there is no account-based recovery.

3. Functional requirements

ID	Requirement	Priority / State
FR-01	Display home, about, report and tracking sections with navigation.	Must / Implemented
FR-02	Accept reporter role: Student, Indigene or Resident.	Must / Implemented
FR-03	Accept community: Akungba, Iwaro, Etioro, Ayegunle or Other.	Must / Implemented
FR-04	Accept issue category: Security, Accommodation, Sanitation, Infrastructure, Harassment or Other.	Must / Implemented

FR-05	Accept issue description between 15 and 2,000 characters.	Must / Implemented
FR-06	Validate submitted fields against allowed values on the server.	Must / Implemented
FR-07	Generate an IRO-XXXXXXXX random hexadecimal tracking reference after successful submission.	Must / Implemented
FR-08	Store reference, role, community, category, description, status and creation timestamp in SQLite.	Must / Implemented
FR-09	Initialize a new report with status Pending.	Must / Implemented
FR-10	Retrieve report reference, category, community, status and submission time when a valid reference is entered.	Must / Implemented
FR-11	Display a not-found message when no matching report exists.	Must / Implemented
FR-12	Protect POST actions with a session CSRF token.	Must / Implemented
FR-13	Allow authorized staff to sign in, view and update reports.	Future / Not implemented
FR-14	Notify users and enable genuine case resolution.	Future / Not implemented

4. Non-functional requirements

ID	Requirement / acceptance criterion

NFR-01	Usability: public submission and tracking forms must have visible labels and required-field indicators.
NFR-02	Responsive UI: forms and navigation should remain usable on common phone and desktop widths.
NFR-03	Security: use parameterized SQL queries, CSRF validation and HTML output escaping.
NFR-04	Privacy: avoid collecting names, phone numbers and matriculation numbers; warn against identifying details in descriptions.
NFR-05	Data integrity: reject invalid role/community/category values and descriptions outside 15–2,000 characters.
NFR-06	Compatibility: run under PHP 8.0+ with PDO SQLite and modern browsers.
NFR-07	Maintainability: keep application logic in index.php and presentation styles in assets/style.css for the mini-project scope.
NFR-08	Reliability: store reports persistently in SQLite across normal page reloads and process restarts.
NFR-09	Performance target: on a typical local development machine, public pages and tracking queries should usually respond within 2 seconds; not formally benchmarked.
NFR-10	Deployment safety: do not publicly expose the SQLite file; do not use this prototype for real sensitive complaints without further security review.

5. System architecture and component interactions

Architecture: Browser (HTML/CSS forms) → PHP application (index.php) → PDO SQLite driver → SQLite database (iro.sqlite). The response returns HTML to the browser. No JavaScript or third-party API is required for the core workflow.

5.1 Main components

Component	Responsibility

Browser / UI	Displays navigation, form fields, confirmation and tracking result.
PHP request handler	Reads POST action, verifies CSRF, validates inputs, inserts or retrieves records.
SQLite database	Stores reports table and provides reference-based lookup.
Stylesheet	Provides responsive visual styling in assets/style.css.
PHP session	Stores per-session CSRF token.

5.2 Submission interaction

1. Reporter opens the Report Issue section. 2. Reporter selects role, community and category, then enters a description. 3. Browser posts action=submit and CSRF token to index.php. 4. PHP checks token and validates fields. 5. PHP generates a random reference and inserts the record through a prepared SQL statement. 6. The page displays the tracking reference. Failed validation displays an error without inserting a report.

5.3 Tracking interaction

1. Reporter opens Track Report. 2. Reporter enters an IRO-XXXXXXXX reference. 3. Browser posts action=track and CSRF token. 4. PHP queries SQLite with a prepared statement. 5. If found, the page shows reference, category, community, Pending status and creation time. Otherwise, a not-found message is displayed.

6. Data requirements

Field	Type / constraint	Purpose
id	INTEGER primary key	Internal row identifier
reference	TEXT UNIQUE	Tracking reference
role	TEXT	Student / Indigene / Resident
community	TEXT	Selected location
category	TEXT	Issue category
description	TEXT, 15–2,000 chars at application level	Report narrative
status	TEXT default Pending	Recorded progress state
created_at	TEXT default CURRENT_TIMESTAMP	Submission timestamp

Data flow and access: report descriptions are stored server-side but are not returned by the public tracking query. A tracking code is a bearer secret for limited metadata; anyone possessing it can look up that metadata.

7. Use cases

Use case	Actor	Precondition	Outcome
UC-01 View information	Student / Indigene / Resident	Page accessible	Homepage and about content displayed
UC-02 Submit issue	Student / Indigene / Resident	Valid form and CSRF token	Report saved; reference shown
UC-03 Track issue	Student / Indigene / Resident	Reference available	Limited report details shown or not-found message

Note: an Administrator actor and report status-update use cases may appear in the proposed-system UML diagram, but these are future requirements, not capabilities of the delivered mini application.

8. Interface requirements

The system uses a single index.php page with anchors #about, #report and #track. Report fields are role, community, category and description. Tracking requires a reference matching IRO- followed by eight hexadecimal characters. CSS is loaded from assets/style.css.

9. Verification and test cases

Test	Input / action	Expected result
T-01	Open index.php	Homepage and navigation render
T-02	Submit valid report with description >=15 chars	Success notice and unique IRO reference
T-03	Submit description shorter than 15 chars	Validation error; no insert
T-04	Submit invalid category through modified POST	Server rejects request
T-05	Track existing reference	Shows category, community, Pending and date

T-06	Track nonexistent valid-format reference	Not-found message
T-07	Submit with invalid CSRF token	Form session error; no action
T-08	Reload after valid submission	Stored report remains retrievable

These are prescribed acceptance tests; execution results are not asserted in this document.

10. Limitations, risks and future work

- No administrator authentication, dashboard or status update capability is implemented.
- Reports remain Pending; the interface must not imply that authorities are reviewing them.
- No emergency dispatch, investigation, moderation, attachments or messaging is provided.
- A public deployment would require HTTPS, rate limiting, stronger privacy controls, restricted database storage, backup, abuse handling and an accountable operating organization.
- The present SQLite file is created beside index.php; deployment must block web access to the file or move it outside the document root.
- Future enhancement: authenticated admin dashboard, role-based access, case audit log and secure notifications.

11. Conclusion

IRO meets the mini-project goal of demonstrating a simple reporting and tracking interface for AAUA students and surrounding communities. It illustrates the main browser, PHP and database interactions while keeping administrative resolution features clearly identified as future work.

Appendix A: Project files

index.php: PHP controller, HTML interface and database initialization. assets/style.css: responsive styling. README.md: installation and scope notes. .gitignore: excludes local database artifacts.

Appendix B: Related submissions

The Ishikawa diagram and UML use-case diagram are separate project artifacts. Any administrator functionality shown in a conceptual diagram should be marked “proposed extension” to match the delivered implementation.